

Technical Specification: Serverless Browser-Based OTC Swap Engine (rod-web-swap)

A cryptographic and architectural analysis of rod-web-swap native browser sandbox execution.

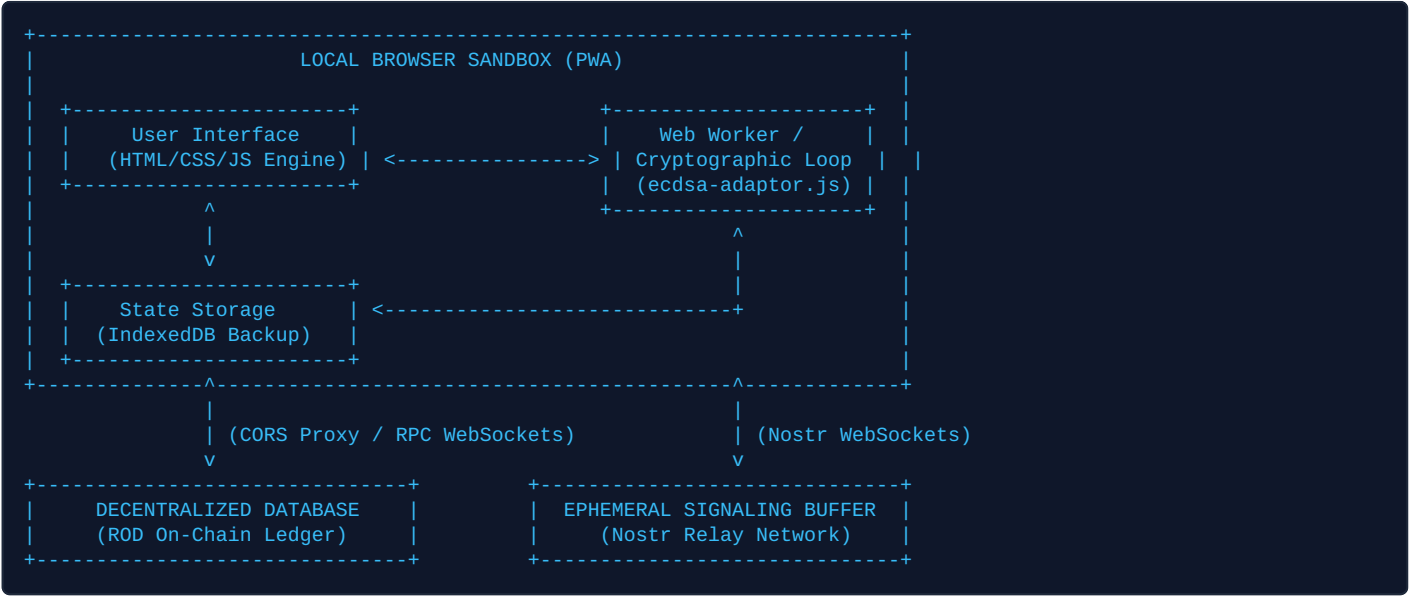
This document provides a rigorous cryptographic and architectural analysis of rod-web-swap, a serverless, non-custodial OTC swap engine built to operate natively within standard web browser sandboxes.

1. SYSTEM TOPOLOGY & ARCHITECTURAL DESIGN

The rod-web-swap OTC swap engine separates **custody of funds** from **market coordination** through a three-tier architecture:

- 1. **Local Execution Sandbox (PWA Client):** Runs directly in the browser. It compiles and evaluates cryptography routines natively in JavaScript using standard secp256k1 curves, managing private keys within local session memory.
- 2. **Persistent Orderbook & Trade Settlement Layer (ROD Blockchain):** Acts as an on-chain, decentralized database. It is a shared state machine that logs active orders, trading indices, and trade history natively.
- 3. **Ephemeral Handshake Signaling Layer (Nostr Network):** Direct peer-to-peer signaling handled via Nostr WebSockets (js / otc-nostr.js). This layer transmits public nonces, parameters, and partial signature payloads.

System Architecture Diagram



2. CRYPTOGRAPHIC ENGINE & THE "VANILLA HACK"

The core atomic settlement primitive of the OTC swap engine relies on **ECDSA Adaptor Signatures** (Scriptless Scripts), mapping the discrete logarithm problem over the secp256k1 elliptic curve of prime order q with base generator point G .

The Mathematical Protocol

Given Alice (holding the secret witness t) and Bob (holding private key x and public key $X = xG$), the protocol executes across four functions:

```

Alice (Witness t, Statement T = tG)                                Bob (Private Key x, Public Key X = xG)
=====
-- Message m, Statement T -->

Compute k in Z_q
R_prime = kG
R = kT
r = R_x mod q
s_prime = k^-1 * (H(m) + r*x) mod q

<-- Pre-Signature (R_prime, s_prime) --

Verify :
s_prime * R_prime == H(m)*G + r*X
If valid, Alice adapts :
s = s_prime * t^-1 mod q

<-- Broadcasts standard signature (r, s) --

Extracts witness :
t = s^-1 * s_prime mod q

```

The "Vanilla" Implementation Strategy

Standard binary deployments of ECDSA rely on deterministic nonce generation (RFC 6979) inside a compiled "black box" (e.g., `vanilla libsecp256k1`) to prevent nonce reuse attacks. This black-box configuration restricts the manual injection of the external adaptor point T .

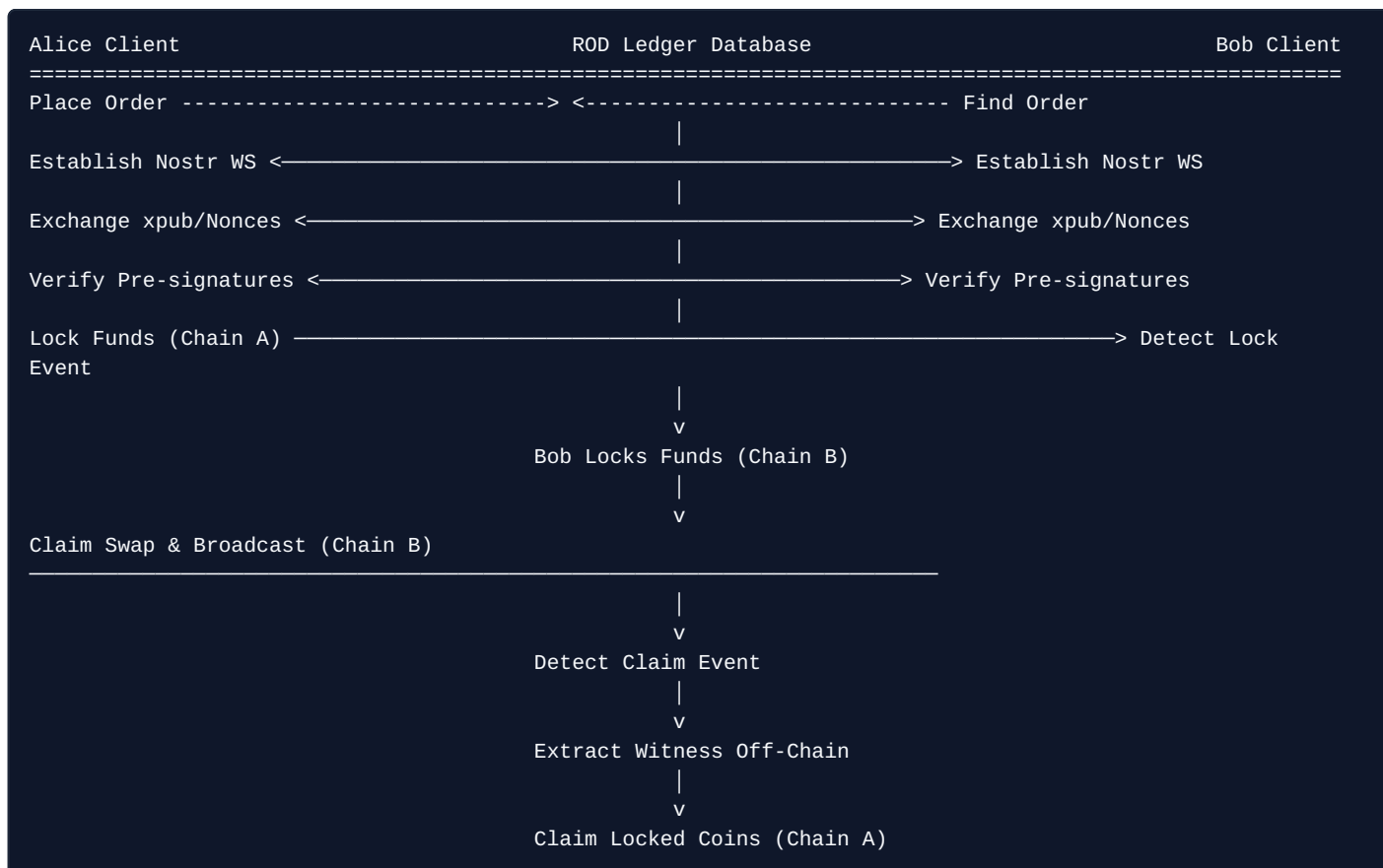
The OTC swap engine implements a **Vanilla math override** in raw JavaScript (`js/ellipticcurve.js`, `js/ecdsa-adaptor.js`), bypassing the need to compile the zero-knowledge `libsecp256k1-zkp` library fork. By rewriting the elliptic curve arithmetic in native JavaScript, the engine directly exposes the internal ephemeral scalar variables. This allows the pre-signing routine to manually intercept the double point multiplication:

$$R = k \cdot T = k \cdot (t \cdot G)$$

This lets the engine inject arbitrary statements and compute modular scalar operations ($s' = k^{-1}(H(m) + r \cdot x) \bmod q$) directly in the JavaScript runtime loop, producing valid signatures that on-chain nodes verify using standard ECDSA.

3. NETWORK TOPOLOGY & STATE SYNCHRONIZATION LAYER

To ensure coordination remains serverless and resistant to front-running exploits, the communication architecture of the OTC swap engine utilizes a dual-layer network topology.



4. FORENSIC ANALYSIS & DERIVATION VULNERABILITIES

Unhardened BIP32/44 Key Derivation

To automate multi-index trading, the browser client derives unhardened child public keys using the parent extended public key (*xpub*).

The derivation is calculated as follows:

$$I = \text{HMAC-SHA512}(\text{Key} = c_{\text{par}}, \text{Data} = K_{\text{par}} || i)$$

Let I_L represent the first 32 bytes of I , and I_R represent the remaining 32 bytes. The child public key K_i is computed as:

$$K_i = I_L \cdot G + K_{\text{par}}$$

Mathematical Proof of Key Compromise

Because unhardened public derivation relies entirely on public variables (K_{par} , c_{par} , and index i), exposing a single child private key k_i alongside the parent *xpub* completely compromises the parent private key k_{par} .

$$k_{\text{par}} = (k_i - I_L) \bmod q$$

Once k_{par} is computed, the attacker derives the extended private key (*xprv*), compromising every child key in that unhardened derivation branch.

Forensic Impact of On-Chain xpub Publishing

The architecture publishes the master Extended Public Key (*xpub*) directly to the ROD blockchain to power non-interactive orderbooks and synchronization indices. This creates a distinct partition in the privacy model:

- **Peer Linkability (Global Forensics):** Because the *xpub* is written to a public database, any observer can pre-calculate all child keys and matching 2-of-2 multisignature lock scripts. This allows investigators to correlate and map the entire multi-chain trading history.

- **Cryptographic Unlinkability (Inter-Chain Privacy):** Because the final adapted signatures contain no shared hash value or static preimages, the mathematical connection between the assets swapped on Chain A and Chain B remains unprovable. The swap looks on-chain like a standard, single-signature or vanilla 2-of-2 multisig transfer, decoupling transactions across different ledgers.

5. MICROARCHITECTURAL TIMING ANALYSIS & BROWSER SANDBOXING

Executing cryptographic operations inside browser sandboxes requires evaluating microarchitectural side channels and the vulnerabilities of non-constant-time math libraries.

Side-Channel Vulnerabilities in jsbn.js

The legacy `jsbn.js` library executes variable-time modular operations, leaving the execution context vulnerable to timing analysis:

1. **Montgomery Exponentiation:** The main multiplication loop includes conditional branches that scale with the bit-length of the exponent, leaking bits of the private key during signing.
2. **Modular Inversion:** The binary GCD loop runs for as long as the lowest bit of the secret value x is zero, leaking the weight and structure of the scalar.
3. **BigNum Comparison (`bnCompareTo`):** The comparison loop returns early the moment a limb differs:

```
while (--i >= 0) {
  if ((r = this[i] - a[i]) != 0) return r; // Early-return leak
}
```

This allows an attacker to extract values limb-by-limb by measuring execution latency.

Browser Countermeasures and Timer Coarsening

Modern browser engines coarsen high-resolution timers (`performance.now()`) to 5 microseconds (isolated) and 100 microseconds (non-isolated) to mitigate side-channel cache attacks like Spectre and Meltdown.

However, attackers can construct high-resolution software timers inside the browser sandbox by using infinite loop counters inside dedicated Web Workers or exploiting atomic operations on `SharedArrayBuffer`. Over thousands of swap trials, statistical analysis can reconstruct private keys by correlating this timing data.

To mitigate these risks, systems should transition legacy bignum operations to constant-time WebCrypto APIs, which compile natively to constant-time libraries like BoringSSL.

Progressive Web App (PWA) State Resilience

To survive tab closures or cellular disconnects mid-swap, the `rod-web-swap` PWA utilizes persistent browser storage and service worker recovery routines:

Swap State	Persisted Parameters	Service Worker Auto-Recovery Protocol
INIT	Derivation index, asset details, public nonces	Idle; await user lock transaction.
LOCKED	Lock tx hash, target script, pre-signatures	Active; poll indexer and Nostr signaling for swap confirmation.
PRE_SIGNED	Ephemeral nonce points R' , pre-signature s'	Verify and adapt signature once counterparty lock is confirmed on-chain.
ADAPTED	Full signature s , extracted secret witness t	Attempt claim broadcast; queue automatic retry on disconnect.
SETTLED	Settled transaction hashes, performance metrics	Purge private variables and session memory; write swap event to log history.

6. IMPLEMENTATION & TEST HARNESS VERIFICATION

The automated refund state transitions are validated by your testing harness (`tests/e2e-swap-test.js`), which stubs out the Nostr signaling networks and blockchain RPC WebSockets to simulate counterparty dropouts.

```
// Verification flow based on tests/e2e-swap-test.js
describe("Atomic Swap Test Suite", () => {
  it("should recover principal via timelocked refund if counterparty goes offline", async () => {
    // Initialize mock environments
    const mockNetwork = new MockBlockchain();
    const mockNostr = new MockNostrRelay();

    const alice = new SwapEngine(aliceConfig, mockNetwork, mockNostr);
    const bob = new SwapEngine(bobConfig, mockNetwork, mockNostr);

    // Alice places her order and locks her funds on Chain A
    await alice.placeOrder();
    await alice.lockFunds();

    // Simulate Bob disconnecting mid-swap
    mockNostr.disconnectPeer('Bob');
    bob.terminate();

    // Assert Alice transitions to recovery state
    assert.equal(alice.getSwapState(), 'WAITING_FOR_RETRY_OR_REFUND');

    // Advance block height past the CSV timelock limit (e.g., 144 blocks)
    await mockNetwork.mineBlocks(144);

    // Trigger recovery loop
    await alice.pollAndRecover();

    // Assert refund transaction has been broadcast
    const refundTx = mockNetwork.getMempool().find(tx => tx.input === alice.lockTxId);
    assert.ok(refundTx, "Refund transaction was not broadcast");
    assert.equal(alice.getBalance(), STARTING_BALANCE, "Alice failed to recover her principal");
  });
});
```

Works Cited

1. ECDSA Adaptor Signature - Cyphertalk, accessed July 19, 2026, <<https://muens.io/ecdsa-adaptor-signature/>>
2. Universally Composable Adaptor Signatures, accessed July 19, 2026, <<https://publ.sec.uni-stuttgart.de/gerhartrauschschroeder-iacr-2025-1363.pdf>>
3. Ask HN: What are you working on? - Hacker News, accessed July 19, 2026, <<https://news.ycombinator.com/item?id=25783152>>
4. TRC102: Hierarchical Deterministic Wallets · Issue #102 · tronprotocol/tips - GitHub, accessed July 19, 2026, <<https://github.com/tronprotocol/tips/issues/102>>
5. Annotations and Discussions on Key Derivation | Joy's Personal Website, accessed July 19, 2026, <<https://joywang.co/2022/12/29/2022-12-29-key-derivation/>>
6. Mastering Bitcoin - Programming The Open Blockchain PDF - Scribd, accessed July 19, 2026, <<https://www.scribd.com/document/362192484/Mastering-Bitcoin-Programming-the-Open-Blockchai-pdf>>
7. HD Wallet : r/Bitcoin - Reddit, accessed July 19, 2026, <https://www.reddit.com/r/Bitcoin/comments/1rhl8xr/hd_wallet/>
8. What is a Point Time Locked Contract (PTLC)? - Nervos Network, accessed July 19, 2026, <https://www.nervos.org/fr/knowledge-base/what_is_a_point_time_locked_contract>
9. Post-Quantum Adaptor Signature for Privacy-Preserving Off-Chain Payments - Erkan Tairi, accessed July 19, 2026, <<https://erkantairi.com/slides/ias-fc.pdf>>
10. Two-Party Adaptor Signatures from Identification Schemes | Request PDF - ResearchGate, accessed July 19, 2026, <https://www.researchgate.net/publication/351268761_Two-Party_Adaptor_Signatures_from_Identification_Schemes>
11. DLT Interoperability and More - Privacy-Preserving Cross-Chain Atomic Swaps (Part 2: Adaptor Signatures), accessed July 19, 2026, <<https://rafaelbelchior.medium.com/dlt-interoperability-and-more-%EF%B8%8F-26-privacy-preserving-cross-chain-atomic-swaps-part-2-adaptor-354de6f28bd6>>
12. What Is Timing / Cache Side-Channel Fingerprint | Scrappey, accessed July 19, 2026, <<https://scrappey.com/qa/anti-bot/what-is-timing-attack-fingerprinting>>
13. Cryptographic Side-Channels (Timing Leaks) in JSBN · Issue #43 - GitHub, accessed July 19, 2026, <<https://github.com/andyperlitch/jsbn/issues/43>>
14. Drive-by Key-Extraction Cache Attacks from Portable Code, accessed July 19, 2026, <<https://cs-people.bu.edu/tromer/drivebycache/>>